

SQL vs MongoDB

SQL

MongoDB

INT

Int32

BIGINT

Int64 (Long)

FLOAT

Double

DECIMAL

Decimal128

VARCHAR

String

CHAR

String

TEXT

String

BOOLEAN

Boolean

DATE



Mongoose crude

▼	BOOLEAN	Boolean
	DATE	Date
	TIMESTAMP	Timestamp / Date
	BLOB	Binary
	Table	Collection
	Row	Document
	Column	Field

MongoDB BSON Data Types

✦ Upgr

Data Type	BSON Type Number	Size	Use	📄
String	2	Variable	Name, Email, Address	
Int32	16	4 bytes	Age, Marks	
Int64 (Long)	18	8 bytes	Large IDs, Big Numbers	
Double	1	8 bytes	Decimal approximation	
Decimal128	19	16 bytes	Money, Salary	
Boolean	8	1 byte	true / false	
Date	9	8 bytes	DOB, CreatedAt	
Timestamp	17	8 bytes	Replication, Internal	
ObjectId	7	12 bytes	Default <code>_id</code>	

Mongoose crude

ObjectId	7	12 bytes	Default <code>_id</code>	
Array	4	Variable	Skills, Subjects	
Object (Document)	3	Variable	Nested Documents	
Null	10	-	Null values	
Binary	5	Variable	Files, Images	
Regular Expression	11	Variable	Regex	
JavaScript	13	Variable	JS Code (Rare)	
MinKey	-1	-	Internal	
MaxKey	127	-	Internal	

Mongoose crude

```
try.js > main
1  const mongoose = require("mongoose");
2
3  async function main() {
4    try {
5      await mongoose.connect("mongodb://127.0.0.1:27017/mytest");
6      console.log("Connected");
7      console.log("Database:", mongoose.connection.name);
8      // All Collections
9      const collections = await mongoose.connection.db.listCollections().toArray();
10     // console.log(collections);
11     collections.forEach((col) => {
12       console.log(col.name);
13     });
14     mongoose.connection.close();
15   } catch (err) {
16     console.log(err);
17   }
18 }
```

2. Database Delete

⌘ JavaScript

```
const mongoose = require("mongoose");

async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

  await mongoose.connection.dropDatabase();

  console.log("Database Deleted");

  mongoose.connection.close();
}

main();
```

// JavaScript

```
const mongoose = require("mongoose");

async function main() {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/mytest");
    console.log("Connected");

    const admin = mongoose.connection.db.admin();

    const dbs = await admin.listDatabases();

    dbs.databases.forEach((db) => {
      console.log(db.name);
    });

    mongoose.connection.close();
  } catch (err) {
    console.log(err);
  }
}

main();
```



3. Collection Rename

⌞ JavaScript

```
const mongoose = require("mongoose");

async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

  await mongoose.connection.db
    .collection("users")
    .rename("students");

  console.log("Collection Renamed");

  mongoose.connection.close();
}

main();
```


4. Collection Delete

⌞ JavaScript

```
const mongoose = require("mongoose");

async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

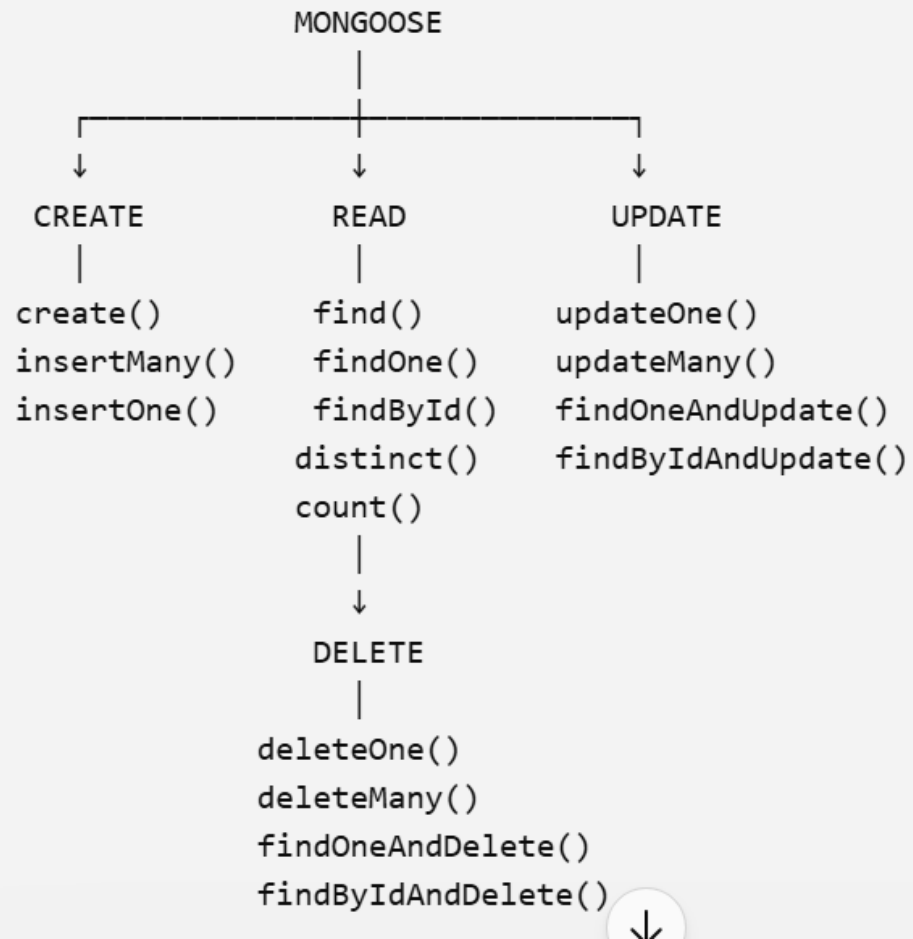
  await mongoose.connection.db
    .collection("users")
    .drop();

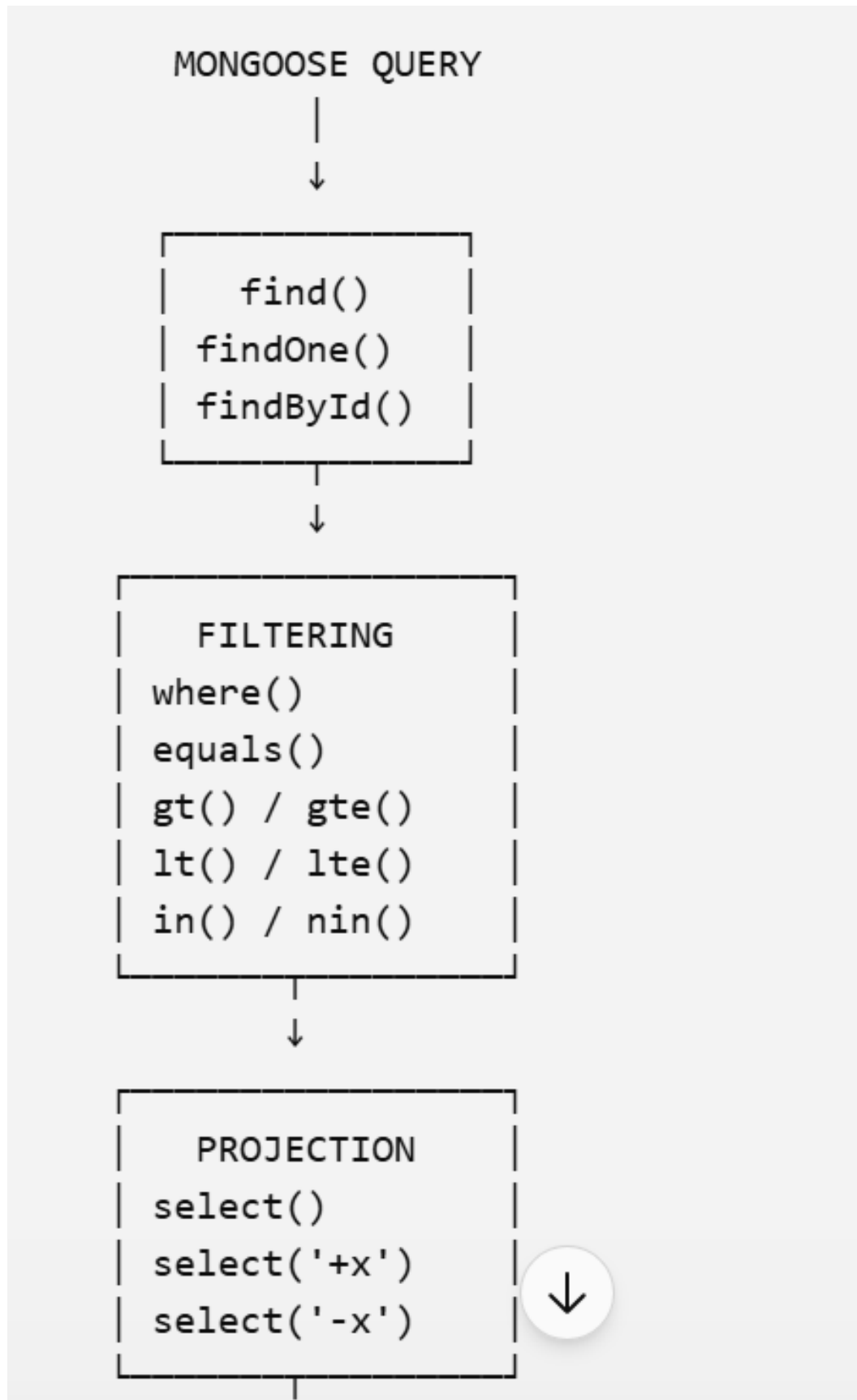
  console.log("Collection Deleted");

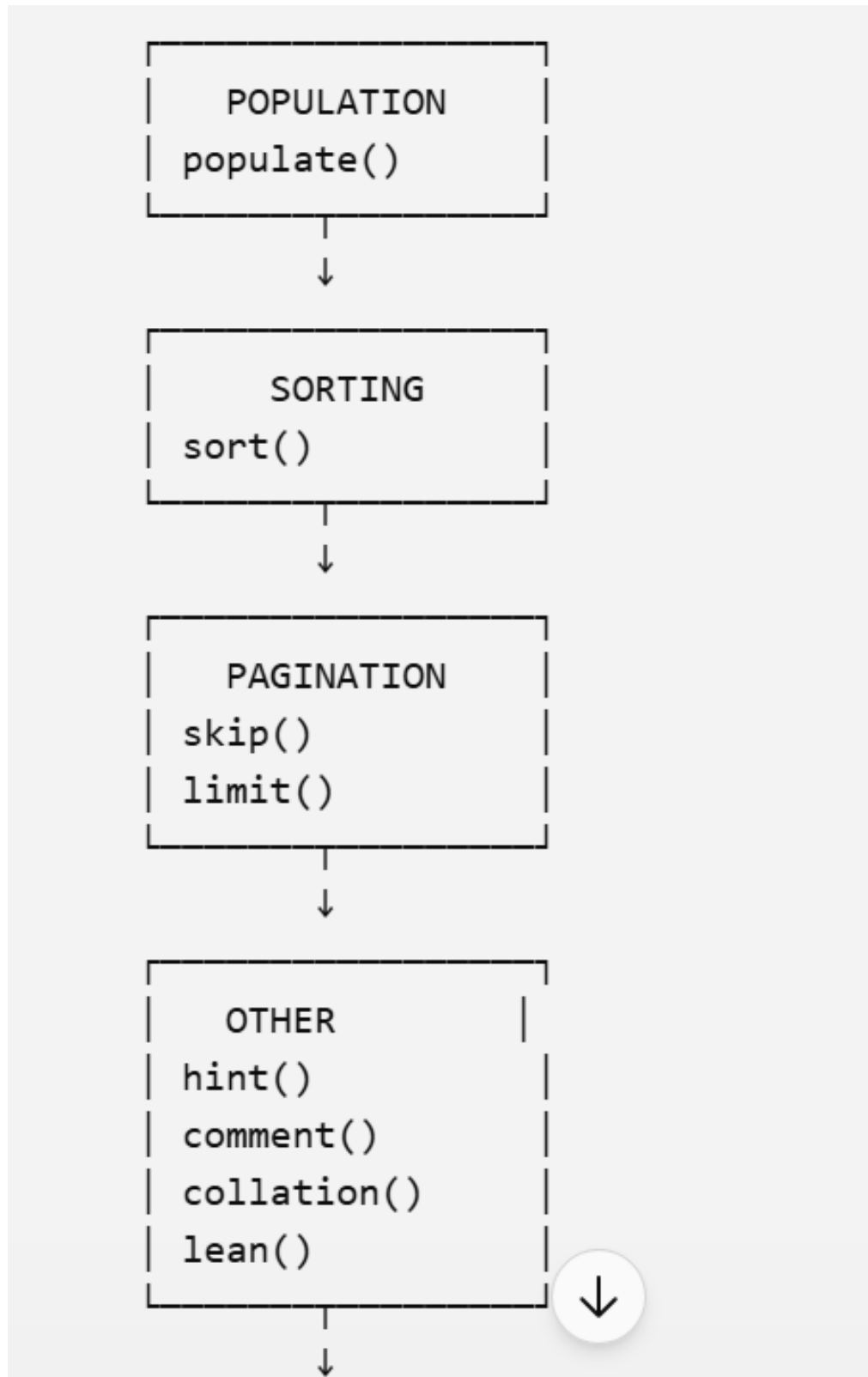
  mongoose.connection.close();
}

main();
```

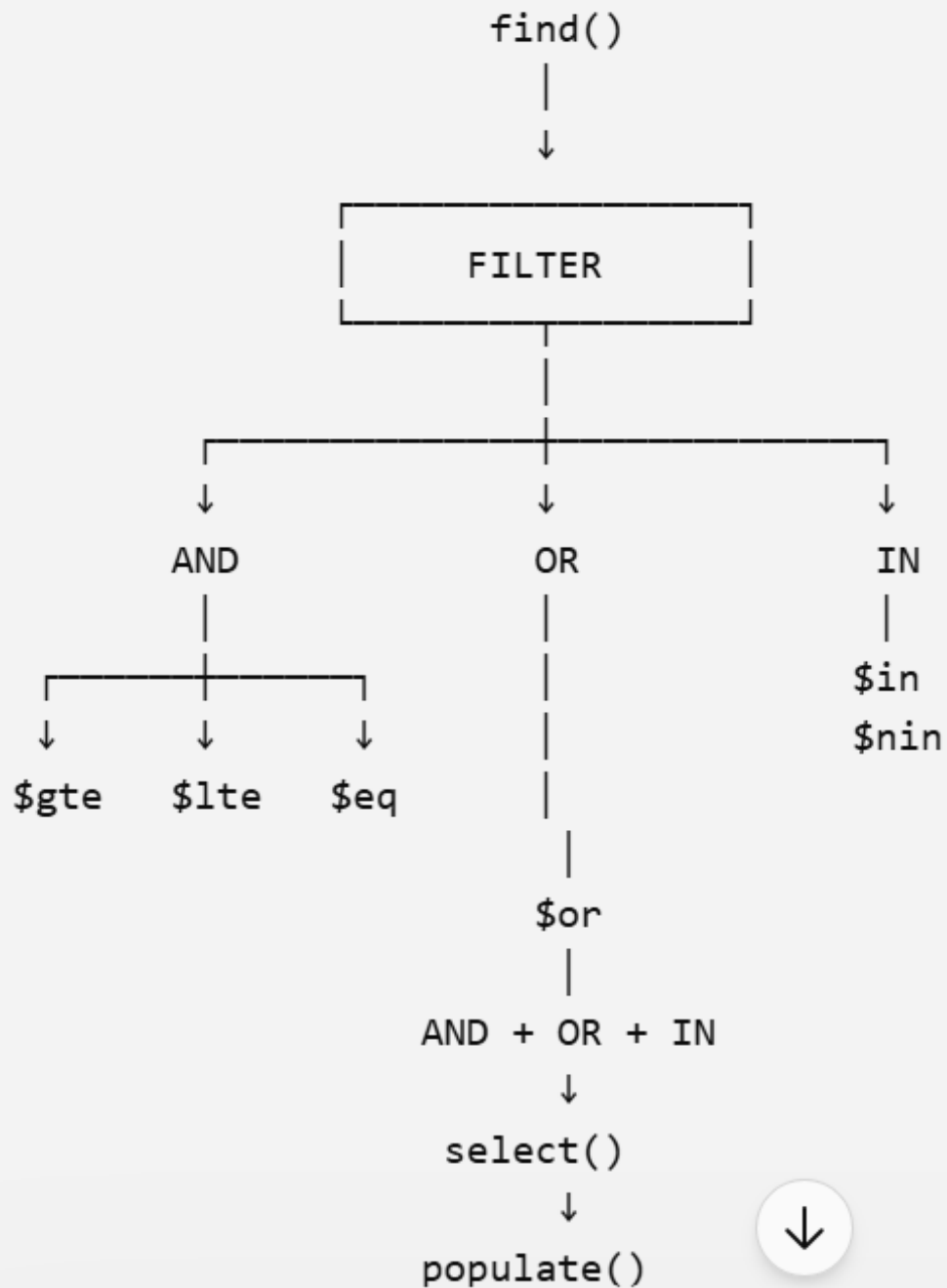
Query methods ko **CRUD-wise** yaad karna interview ke liye better hai:







🔥 Final Mongoose FIND hierarchy



d10-306aec494223

```
↓  
select()  
↓  
populate()  
↓  
sort()  
↓  
skip()  
↓  
limit()  
↓  
lean()  
↓  
await  
↓  
RESULT
```

Sabse important shortcut:

`find()` ke baad pehle **data filter karo** (`AND/OR/IN`), phir **fields choose** (`select`), phir **related data** (`populate`), phir **order** (`sort`), phir **pagination** (`skip/limit`), aur end me read performance ke liye `lean()` .

// JavaScript

```
const mongoose = require("mongoose");

const firstSchema = new mongoose.Schema(
  {},
  { strict: false }
);

const secondSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      unique: true
    }
  },
  { strict: false }
);

const firstModel = mongoose.model(
  "firstModel",
  firstSchema,
  "firstModel"
);
```



```
const secondModel = mongoose.model(  
  "secondModel",  
  secondSchema,  
  "secondModel"  
);  
  
async function main() {  
  
  try {  
  
    console.time("api");  
  
    await mongoose.connect(  
      "mongodb://localhost:27017/test"  
    );  
  
    console.timeEnd("api");  
  
    console.log("connect successful");  

```




```
const data = await firstModel
  .find({
    age: {
      $lte: 55,
      $gte: 23
    },
    city: "nalkheda"
  })
  .select("name age city")
  .sort({ age: -1 })
  .skip(2)
  .limit(5)
  .lean();

console.log(data);

}
catch (err) {
  console.log(err);
}

}
```



1. `where()` + `gte()` + `lte()`

Tumhari query ko is tarah bhi likh sakte ho:

`</>` JavaScript

```
const data = await firstModel
  .find()
  .where("age").gte(23).lte(55)
  .where("city").equals("nalkheda")
  .select("name age city")
  .sort({ age: -1 })
  .skip(2)
  .limit(5)
  .lean();
```

2. populate()

Agar ek collection me reference hai:

 JavaScript 

```
const data = await Student
  .find()
  .populate("department")
  .select("name age department")
  .sort({ age: -1 })
  .limit(5)
  .lean();
```

populate() ka use reference wale document ko automatically fetch karne ke liye hota

l-306aec494223

3. `exists()`

`</>` JavaScript

```
const data = await firstModel
  .find()
  .where("email").exists(true);
```

Meaning:

email field present hona chahiye

4. `in()`

`</>` JavaScript

```
const data = await firstModel
  .find()
  .where("city")
  .in(["Indore", "Bhopal", "Ujjain"]);
```



5. regex

Search ke liye:

`</>` JavaScript

```
const data = await firstModel.find({
  name: {
    $regex: "^A",
    $options: "i"
  }
});
```

Meaning:

Name A se start ho

Agar unique values chahiye:

`</> JavaScript`

```
const cities = await firstModel.distinct("city");

console.log(cities);
```

Example:

```
["Indore", "Bhopal", "Ujjain", "Dewas"]
```

7. `countDocuments()`

Kitne documents match karte hain:

`</> JavaScript`

```
const count = await firstModel
  .countDocuments({
    city: "nalkheda"
  });

console.log(count);
```

```
// JavaScript
```

```
const data = await firstModel
  .find({
    age: { $gte: 23 },

    $or: [
      { city: "Indore" },
      { city: "Bhopal" }
    ]
  })
  .select("name age city")
  .sort({ age: -1 })
  .skip(2)
  .limit(5)
  .lean();
```



7. AND + OR + IN — Advanced



Suppose question:

Age 23–55 ke beech ho **AND** city Indore/Bhopal/Ujjain me ho **AND** marks 60 se zyada ho.

</> JavaScript



```
const data = await firstModel
  .find({
    age: {
      $gte: 23,
      $lte: 55
    },
    city: {
      $in: [
        "Indore",
        "Bhopal",
        "Ujjain"
      ]
    },
  })
```




```
    ],  
    },  
    marks: {  
      $gt: 60  
    }  
  })  
  .select("name age city marks")  
  .sort({ age: -1 })  
  .skip(2)  
  .limit(5)  
  .lean();
```

Yahan top-level conditions automatically **AND** hain:

```
age 23-55  
  AND  
city IN [Indore, Bhopal, Ujjain]  
  AND  
marks > 60
```



Mongoose crude

```
1  const mongoose = require("mongoose");
2  const userSchema = new mongoose.Schema({
3
4    phone: {
5      type: String,
6      required: true,
7      unique: true,
8      match: /^[789][0-9]{9}$/
9    },
10
11   password: {
12     type: String,
13     required: true,
14     minlength: 16,
15     maxlength: 16,
16     match: /^(?=[A-Z])(?=[a-z])(?=[A-Z])(?=[0-9])(?=[@#$%])[A-Za-z0-9@#$%]{16}$/
17   },
18
19   dob: {
20     type: String,
21     required: true,
22     match: /^(0[1-9] | [12][0-9] | 3[01])\/(0[1-9] | 1[0-2])\[0-9]{4}$/
23   },
24 });
25
26
27 const User = mongoose.model("User", userSchema);
```

</> JavaScript

```
async function main() {  
  
    await mongoose.connect(  
        "mongodb://localhost:27017/test"  
    );  
  
    try {  
  
        const user = await User.create({  
  
            phone: "9876543210",  
  
            password: "Abcdefgh12345@#$",  
  
            dob: "18/08/2002"  
  
        });  
  
        console.log(user);  
  
    }  
    catch (err) {
```



</> JavaScript

```
const users = await User.insertMany([

  {
    phone: "9876543210",
    password: "Abcdefgh12345@#$",
    dob: "18/08/2002"
  },

  {
    phone: "8765432109",
    password: "Xabcdef1234567@$",
    dob: "25/12/2001"
  },

  {
    phone: "7654321098",
    password: "Pqwerty1234567@#",
    dob: "05/01/2003"
  }

]);

console.log(users);
```



```
> JS query.js > userSchema > city > enum
1  const userSchema = new mongoose.Schema({
2    username: {
3      type: String,
4      required: true,
5      minlength: 3,
6      maxlength: 30,
7      trim: true,
8      match: /^[A-Za-z0-9_]+$/,
9      unique: true,
10     index: true
11   },
12
13   email: {
14     type: String,
15     required: true,
16     trim: true,
17     lowercase: true,
18     unique: true
19   },
20 }
```

```

    },
    password: {
      type: String,
      required: true,
      minlength: 16,
      maxlength: 16,
      match: /^(?=[A-Z])(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@#$])[A-Za-z0-9@#${16}$/
    },
    city: {
      type: String,
      default: "Indore",
      enum: ["Indore", "Bhopal", "Ujjain", "Dewas"]
    }
  });

```

VALIDATION

- └─ required
- └─ minlength
- └─ maxlength
- └─ enum
- └─ match
- └─ validate

TRANSFORMATION

- └─ trim
- └─ lowercase
- └─ uppercase

DATABASE / INDEX

- └─ unique
- └─ index

DEFAULT

- └─ default

Upgra

Ye distinction interview me "Mongoose schema ke String type ke options kaun-kaun se hain?" wale question me kaafi useful hai.



Mongoose crude

interview/practical ke liye ye main string options hain.

#	Option	Purpose	Example	Result	
1	type	Data type define karta hai	type: String	"Tarun"	
2	required	Field compulsory banata hai	required: true	Value na dene par validation error	
3	minlength	Minimum string length	minlength: 3	"Ta" ❌	
4	maxlength	Maximum string length	maxlength: 50	51 chars ❌	
5	trim	Beginning/end ke spaces remove karta hai	trim: true	" Tarun " → "Tarun"	
6	lowercase	Value ko lowercase karta hai	lowercase: true	"TARUN" → "tarun"	
7	uppercase	Value ko uppercase karta hai	uppercase: true	"tarun" → "TARUN"	
8	enum	Sirf given values allow karta hai	enum: ["Male", "Female"]	"other" ❌	
9	match	Regex pattern validate karta hai	match: /^[A-Za-z]+\$	"Tarun123" ❌	
10	validate	Custom validation condition	validate: fn	Apni condition laga sakte ho	
11	unique	Duplicate value ke liye unique index	unique: true	Same email duplicate ❌	
12	index	Field par index create karta hai	index: true	Searching/querying faster ho sakti hai	
13	default	Value na dene par default value	default: "Unknown"	Missing → "Unknown"	

Mongoose crude

14	<code>alias</code>	Field ka alternate name define karta hai	<code>alias: "userName"</code>	<code>name</code> ko <code>userName</code> se access
15	<code>immutable</code>	Set hone ke baad change prevent karta hai	<code>immutable: true</code>	Update me change nahi hoga
16	<code>select</code>	Query result me field by default include/exclude	<code>select: false</code>	Password normally return nahi hoga
17	<code>get</code>	Read karte waqt value transform	<code>get: v => v.trim()</code>	Output transform
18	<code>set</code>	Save karte waqt value transform	<code>set: v => v.trim()</code>	Save se pehle transform

</> JavaScript

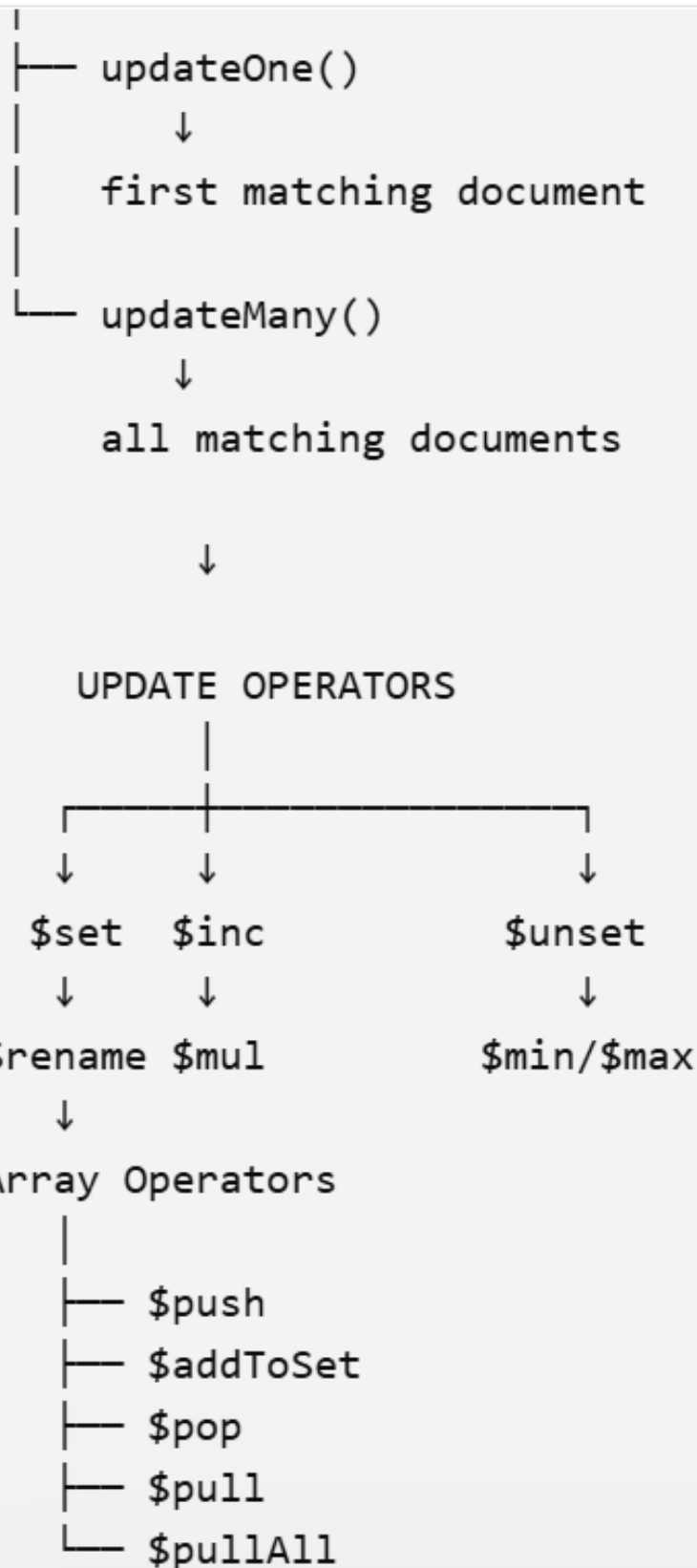
```
const userSchema = new mongoose.Schema({

  name: {
    type: String,
    required: true,
    minlength: 3,
    maxlength: 50,
    trim: true,
    lowercase: true,
    uppercase: false,
    enum: ["Rahul", "Amit", "Tarun"],
    match: /^[A-Za-z ]+$/,
    unique: true,
    index: true,
    default: "Unknown"
  }

});
```



306aec494223



```
</> JavaScript

await Model.updateOne(
  { filter },
  { update }
);
```

Example

```
</> JavaScript

const result = await firstModel.updateOne(
  { name: "Rahul" },
  {
    $set: {
      age: 25,
      city: "Indore"
    }
  }
);

console.log(result);
```



2. `updateMany()`

Saare matching documents ko update karta hai.

`</>` JavaScript

```
const result = await firstModel.updateMany(
  { city: "Indore" },
  {
    $set: {
      status: "active"
    }
  }
);

console.log(result);
```

Meaning:

```
city = Indore
↓
ALL matching documents
↓
status = active
```



3. Update operators

`$set` ke alawa bahut operators hain.

Operator	Purpose	Example
<code>\$set</code>	Value set/update	<code>\$set: {age: 25}</code>
<code>\$unset</code>	Field remove	<code>\$unset: {age: 1}</code>
<code>\$inc</code>	Number increase/decrease	<code>\$inc: {age: 1}</code>
<code>\$mul</code>	Number multiply	<code>\$mul: {salary: 2}</code>
<code>\$min</code>	Smaller value preserve	<code>\$min: {marks: 50}</code>
<code>\$max</code>	Larger value preserve	<code>\$max: {marks: 90}</code>
<code>\$rename</code>	Field rename	<code>\$rename: {name:"fullName"}</code>
<code>\$push</code>	Array me add	<code>\$push: {skills:"JS"}</code>
<code>\$addToSet</code>	Array me unique value add	<code>\$addToSet:{skills:"JS"}</code>

<code>\$addToSet</code>	Array me unique value add	<code>\$addToSet:{skills:"JS"}</code>
<code>\$pop</code>	Array ka first/last remove	<code>\$pop:{skills:1}</code>
<code>\$pull</code>	Array se matching value remove	<code>\$pull:{skills:"Java"}</code>
<code>\$pullAll</code>	Multiple array values remove	<code>\$pullAll:{skills:["Java","C"]}</code>

5. \$inc

Age 1 increase:

`</>` JavaScript

```
await firstModel.updateMany(
  { city: "Indore" },
  {
    $inc: {
      age: 1
    }
  }
);
```

Agar:

```
Rahul → 23
Amit → 25
Ravi → 30
```

6. \$unset

Field delete karna:

`</>` JavaScript

```
await firstModel.updateOne(
  { name: "Rahul" },
  {
    $unset: {
      city: 1
    }
  }
);
```

`city` field remove ho jayegi.

7. \$rename

</> JavaScript

```
await firstModel.updateMany(
  {},
  {
    $rename: {
      "name": "fullName"
    }
  }
);
```

Before:

</> JavaScript

```
{
  name: "Rahul"
}
```


8. \$min

Suppose current marks:

80

</> JavaScript

```
await firstModel.updateOne(
  { name: "Rahul" },
  {
    $min: {
      marks: 70
    }
  }
);
```

Result:

70

10. Array \$push

Suppose:

`</>` JavaScript

```
{
  name: "Rahul",
  skills: ["C++", "SQL"]
}
```

`</>` JavaScript

```
await firstModel.updateOne(
  { name: "Rahul" },
  {
    $push: {
      skills: "JavaScript"
    }
  }
);
```



Result:

13. Multiple operators ek saath

</> JavaScript

```
await firstModel.updateOne(
  { name: "Rahul" },
  {
    $set: {
      city: "Bhopal"
    },
    $inc: {
      age: 1
    },
    $unset: {
      temporaryField: 1
    }
  }
);
```



Ek hi update me:

1. deleteOne()

Condition se match hone wala first document delete karta hai.

Syntax

JavaScript

```
await Model.deleteOne(  
  { condition }  
);
```

Example

JavaScript

```
const result = await firstModel.deleteOne({  
  name: "Rahul"  
});  
  
console.log(result);
```



2. deleteMany()

Condition se match hone wale saare documents delete karta hai.

`</>` JavaScript

```
const result = await firstModel.deleteMany({  
  city: "Indore"  
});  
  
console.log(result);
```

Agar 20 students ki city "Indore" hai:

```
20 matching  
  ↓  
deleteMany()  
  ↓  
20 deleted
```

3. Filter ke saath delete

Age 50 se greater

</> JavaScript

```
await firstModel.deleteMany({  
  age: {  
    $gt: 50  
  }  
});
```

Age 23–30

</> JavaScript

```
await firstModel.deleteMany({  
  age: {  
    $gte: 23,  
    $lte: 30  
  }  
});
```

4. AND condition

`</>` JavaScript

```
await firstModel.deleteMany({  
  age: {  
    $gte: 23,  
    $lte: 30  
  },  
  city: "Indore"  
});
```

Meaning:

```
age >= 23  
AND  
age <= 30  
AND  
city = Indore
```

5. OR condition

</> JavaScript

```
await firstModel.deleteMany({
  $or: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
});
```

Meaning:

```
city = Indore
OR
city = Bhopal
```


7. findOneAndDelete()

Agar document delete karna hai aur deleted document bhi return chahiye, to:

`</> JavaScript`

```
const deletedUser = await firstModel.findOneAndDelete({  
  name: "Rahul"  
});  
  
console.log(deletedUser);
```

Yahan:

```
find  
↓  
delete  
↓  
deleted document return
```

10. `deleteOne()` vs `deleteMany()`

Point	<code>deleteOne()</code>	<code>deleteMany()</code>
Documents	First matching	All matching
Filter	<code>{}</code>	<code>{}</code>
Return	Delete result	Delete result
<code>deletedCount</code>	Usually <code>0</code> or <code>1</code>	<code>0</code> to many
Use	Single document	Bulk delete

Example:

Specific cities

`</>` JavaScript

```
await firstModel.deleteMany({  
  city: {  
    $in: ["Indore", "Bhopal", "Ujjain"]  
  }  
});
```

Meaning:

```
city = Indore  
OR  
city = Bhopal  
OR  
city = Ujjain
```